NATURAL LANGUAGE PROCESSING

MODULE 3:
WORD LEVEL ANALYSIS

CONTENTS

- **Tokenization**
- **Segmentation**
- **Lemmatization**
- **Edit Distance**
- **Collocations**
- **Porter Stemmer**
- **N-gram Language Model**
- **Morphological Analysis**
- **Derivational and Reflectional Morphology**

- **Segmentation**
- **Tokenization**
- **Morphological Analysis**
- **Derivational and Inflectional Morphology**
- **Porter Stemmer**
- **Lemmatization**
- **Edit Distance**
- **Collocations**
- **N-gram Language Model**

SEGMENTATION

Segmentation

- Word Segmentation
- Intent Segmentation
- Sentence Segmentation
- Topic Segmentation : binary classification problem, where we classify each sentence and determine if it is a boundary sentence.

"[All things are made of **atoms**]. [Little **particles** that move] [around in perpetual **motion**], [attracting each **other**] [when they are a little **distance** apart], [but **repelling**] [upon being **squeezed**] [into **one another**]."

Segmentation

Word Segmentation

Dividing **continuous text into words**
Mainly needed for languages **without spaces** (e.g., Chinese, Japanese, Thai)
Decides **where word boundaries exist**

Example:
NaturalLanguage → Natural | Language

我喜欢自然语言处理 → 我 | 喜欢 | 自然 | 语言 | 处理

Segmentation

Sentence Segmentation

Sentence segmentation (also called sentence boundary detection) involves **dividing a paragraph into individual sentences**.

Example:

> *NLP is interesting. It is widely used.*
> → Two separate sentences

Segmentation

Intent Segmentation

Intent segmentation identifies **the intention or purpose** behind a user's utterance, especially in **dialogue systems and chatbots**.

Example:

> *Book a ticket and cancel my reservation*
> → Intent 1: Book ticket
> → Intent 2: Cancel reservation

Segmentation

Topic Segmentation

Topic segmentation divides a document into **topically coherent segments**.

- It is treated as a **binary classification problem**
- Each sentence is classified as:
  - **Boundary sentence** (topic change)
  - **Non-boundary sentence**

Segmentation

Topic Segmentation

Example:

S1: Artificial Intelligence is transforming many industries through automation.

S2: Machine learning allows systems to improve performance using data.

S3: Deep learning models are widely used in image and speech recognition.

S4: However, training deep learning models requires large datasets and high computational power.

Boundary Statement

S5: Cloud computing provides scalable resources such as storage and processing.

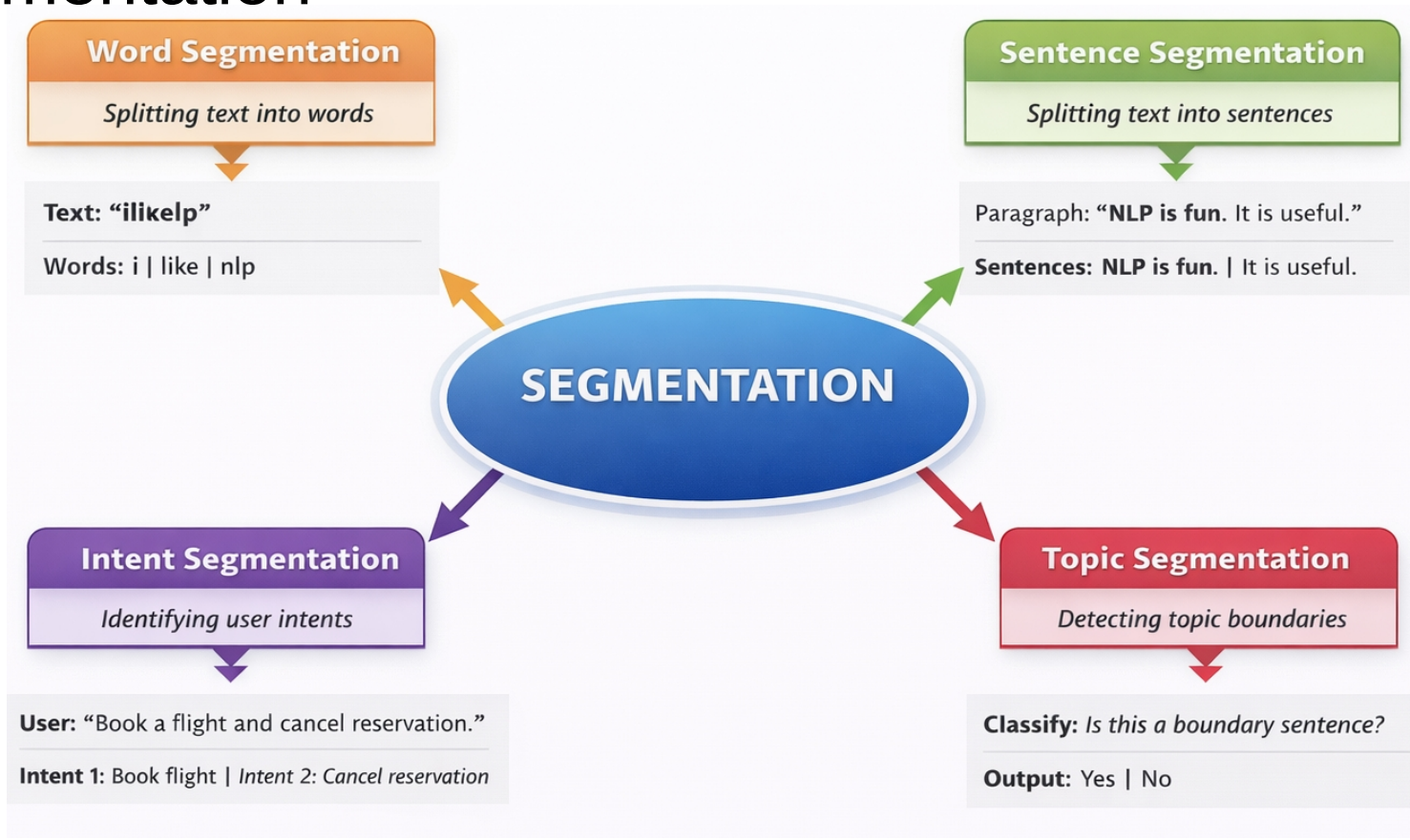
S6: Many organizations use cloud platforms to deploy machine learning models.

Boundary Statement

S7: Data security and privacy remain major concerns in cloud environments.

S8: Governments across the world are introducing regulations to protect user data.

Segmentation



TOKENIZATION

Tokenization

- Splits text into **tokens** (words, numbers, punctuation)
- Works mainly for languages **with spaces** (e.g., English)
- Tokens may include symbols and punctuation

Example:

Natural Language → Natural | Language

NaturalLanguage → NaturalLanguage

Tokenization

- process of breaking a stream of textual data into words, terms, sentences, symbols, or some other meaningful elements called tokens.
- A token may be a word, part of a word or just characters like punctuation.
- In other words, a token is a string with a known meaning.
- Tokenization is the first step in any NLP pipeline.
- A tokenizer breaks unstructured data and natural language text into chunks of information that can be considered as discrete elements.
- The token occurrences in a document can be used directly as a vector representing that document.
- This immediately turns an unstructured string (text document) into a numerical data structure suitable for machine learning.

Types of Tokenization

- White space Tokenization
- Word Tokenization
- Sentence Tokenization
- Subword Tokenization
- Character Tokenization

White Space Tokenization

Splits text on spaces, tabs, and newlines

Example:

Input:
"Hello world! How are you?"

Output:
`["Hello", "world!", "How", "are", "you?"]`

✓ Advantages

- Simple and fast
- No special libraries needed
- Language-agnostic

✗ Limitations

- Doesn't remove punctuation
- Fails with languages without spaces
- Can't handle contractions well

Word Tokenization

Splits text into meaningful words using linguistic rules

Example:

Input:
"Don't worry, it's working!"

Output:
```
["Do", "n't", "worry", ",", "it", "'s",
"working", "!"]
```

Key Features

- Handles punctuation intelligently
- Separates contractions (don't → do + n't)
- Common in NLP libraries (NLTK, spaCy)
- Language-specific rules apply

Best For:

Text analysis, sentiment analysis, part-of-speech tagging

Sentence Tokenization

Splits text into individual sentences

Example:

Input:
"Dr. Smith arrived. She works at MIT. Great!"

Output:

```
[
  "Dr. Smith arrived.",
  "She works at MIT.",
  "Great!"
]
```

Challenges to Handle

- Abbreviations (Dr., Mr., etc.)
- Decimal numbers (3.14)
- Ellipsis (...)
- Quoted text with periods

Best For:

Document summarization, translation, text segmentation

Subword Tokenization

Breaks words into smaller meaningful units

Example (BPE):

Input:
"unhappiness"

Output:
`["un", "happi", "ness"]`

Why Use Subword?

Handles rare/unknown words, reduces vocabulary size, captures morphology (prefix/suffix)

Popular Algorithms

BPE

Byte Pair Encoding • GPT models

WordPiece

Frequency-based • BERT

SentencePiece

Language-agnostic • T5, ALBERT

Character Tokenization

Splits text into individual characters

Example:

Input:
"Hello"

Output:
`["H", "e", "l", "l", "o"]`

- Minimal vocabulary
- No OOV words
- Works with any language

- Very long sequences
- Loses semantic meaning
- Computationally expensive

Best For:

Character-level language models, spelling correction, OCR

Tokenization

Method	Vocab Size	Sequence Length	Best Use Case
White Space	Large	Short	Quick prototypes
Word	Large	Short	Traditional NLP
Sentence	N/A	Very Short	Document processing
Subword	Medium	Medium	Modern LLMs
Character	Tiny	Very Long	Spelling/OCR

NATURAL LANGUAGE PROCESSING
LECTURE 9:

MODULE 3:
WORD LEVEL ANALYSIS

Significance of Tokenization

- **Effective Text Processing:** Tokenization reduces the size of raw text so that it can be handled more easily for processing and analysis.
- **Feature extraction:** Text data can be represented numerically for algorithmic comprehension by using tokens as features in machine learning models.
- **Language Modelling:** Tokenization in NLP facilitates the creation of organized representations of language, which is useful for tasks like text generation and language modelling.
- **Information Retrieval:** Tokenization is essential for indexing and searching in systems that store and retrieve information efficiently based on words or phrases.

Significance of Tokenization

- **Text Analysis:** Tokenization is used in many NLP tasks, including sentiment analysis and named entity recognition, to determine the function and context of individual words in a sentence.
- **Vocabulary Management:** By generating a list of distinct tokens that stand in for words in the dataset, tokenization helps manage a corpus's vocabulary.
- **Task-Specific Adaptation:** Tokenization can be customized to meet the needs of particular NLP tasks, meaning that it will work best in applications such as summarization and machine translation.
- **Preprocessing Step:** This essential preprocessing step transforms unprocessed text into a format appropriate for additional statistical and computational analysis.

MORPHOLOGICAL ANALYSIS

Morphological Analysis

- Morphological analysis is a field of linguistics that studies the structure of words - identifies how a word is produced through the use of morphemes.
- A morpheme is the smallest element of a word that has grammatical function and meaning.

  Lexical and grammatical morphemes

  - A single free morpheme can become a complete word. For instance, a bus, a bicycle etc.
  - A bound morpheme cannot stand alone and must be joined to a free morpheme to produce a word. ing, un, and other bound morphemes are examples.

Roots

The central morphemes in words, which carry the main meaning

Affixes

- Prefixes
- Suffixes
- Infixes

Morphological Analysis

- Inflectional morphology
  - Adds information to a word consistent with its context within a sentence
  - Can be infix or suffix. Will never be prefix.
  - Examples
    - Number (singular versus plural) Walk → walks
    - Case (nominative versus accusative versus…) he, him, his, …
- Derivational morphology
  - Creates new words with new meanings (and often with new parts of speech) - class changing morpheme and class maintaining morpheme
  - Can be prefix or suffix.
  - Examples : parse → parser, repulse → repulsive, child → childhood

PORTER STEMMER

Stemming

- **Stemming** is a method in **text processing** that eliminates prefixes and suffixes from words, transforming them into their fundamental or root form.
- The main objective of stemming is to streamline and standardize words, enhancing the effectiveness of the natural language processing tasks.
- Stemming has drawbacks, including potential impacts on text readability and occasional inaccuracies in determining the correct root form of a word.

Stemming

- Aides in text normalization for easier processing.
- This technique is crucial in tasks like text classification, information retrieval, and text summarization.
- In text mining pipelines it converts raw text data into a structured format for machine processing.
- Stemming is a more heuristic process of removing suffixes to produce a base form.
- Lemmatization conducts a more detailed morphological analysis, often involving part of speech (POS) tagging and mapping output to real word roots contained in dictionaries.

Stemmer

- Stemmers are the algorithms used to reduce different forms of a word to a base form.
- Essentially, they do this by removing specific character strings from the end of word tokens.
- Stemmers thus do not account for prefixes.
- Most stemmers contain a list of common language-specific suffixes against which the algorithm matches input word tokens.
- If the algorithm matches a word to one of the suffixes, and stripping the suffix does not violate pre-specified rules in the algorithm (e.g. character length restrictions), then the algorithm removes the suffix from the word.

Types of Stemmer

- Types of Stemmer
  - **Porter Stemmer :** based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes.
  - Lovin Stemmer : removes the longest suffix from a word then the word is recorded to convert this stem into valid words.
  - Dawson Stemmer : suffixes are stored in the reversed order indexed by their length and last letter
  - Krovetz Stemmer: Convert the plural form of a word to its singular form. Convert the past tense of a word to its present tense and remove the suffix ‘ing’.
  - Xerox Stemmer : Capable of processing extensive datasets and generating valid words, and is language-dependent.

Types of Stemmer

- Types of Stemmer
  - **N-Gram Stemmer :** involves breaking words into segments of length n and then applying statistical analysis to identify patterns
  - Snowball Stemmer: can handle non-English words
  - Lancaster Stemmer : save the rules externally and uses an iterative algorithm
  - Regxp Stemmer : utilizes regular expressions to identify and remove suffixes from words.

Porter Stemmer

- Common algorithm for stemming English.
- The Porter stemming algorithm classifies every character in a given token as either a consonant ("c") or vowel ("v"), grouping subsequent consonants as "C" and subsequent vowels as "V."
- The algorithm thereby represents every word token as a specific combination of consonant and vowel groups.
- Consonant: a letter other than A,E,I,O,U and Y preceded by consonant.
- Vowel: A,E,I,O,U

Porter Stemmer

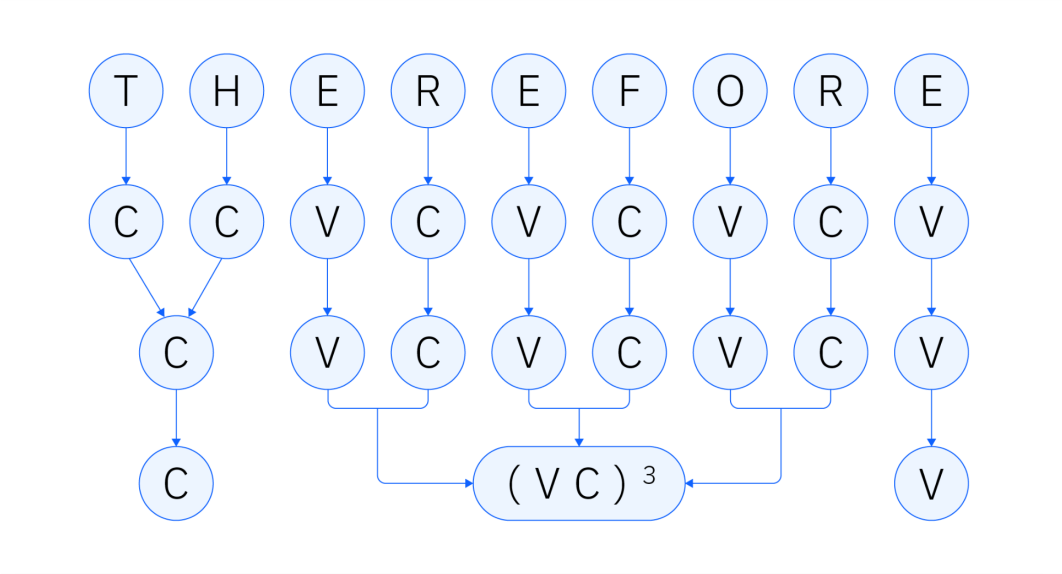
- All words are of the form:

(C) (VC)^m (V)

m = measure of word or word part when represented in the form (VC)

Porter Stemmer

- For example, the word *therefore* is represented as CVCVCVCV, or C(VC)3V, with the exponent representing repetitions of consonant-vowel groups.



Porter Stemmer

- All words are of the form:

  (C) (VC)^m (V)

  m = measure of word or word part when represented in the form (VC)

- For example,
  - m = 0 TREE , BY
  - m = 1 TREES , TROUBLE, OATS
  - m = 2 TROUBLES, OATEN, PRIVATE

Porter Stemmer

- Once enumerated this way, the stemmer runs each word token against a list of rules that specify the ending characters to remove according to the number of vowel-consonant groups in that token.
- Rules for removing the suffixes are of the form:

  (condition) s1 → s2
- This means that if a word ends with a suffix s1, and the stem before s1 satisfies the given condition then s1 is replaced by s2.
- For example:

  If rule is (m>1) ement → null

  Replacement → Repalc

Porter Stemmer

- The condition part may contain

m	The measure of the stem
*char	The stem ends with the char specified
v	The stem contains a vowel
*d	The stem ends with double consonant
*o	The stem ends in CVC (second stem not w,x or y)
&, |, !	May contain expressions with and, or and not

Porter Stemmer

- The Porter Stemmer has 5 rules
- Rule 1: Part A

Sr. No.	Rule	Example
1.	sses → ss	caresses → caress
2.	ies → i	ponies → poni
3.	ss → ss	caress → caress
4.	s → Φ	cats → cat

Porter Stemmer

- Rule 1: Part B

Sr. No.	Rule	Condition Verified	Condition Not Verified
1.	(m>1) eed → ee	agreed → agree	feed → feed
2.	(*V*) ed → ***Φ***	plastered → plaster	bled → bled
3.	(*V*) ing → ***Φ***	motoring → motor	sing → sing

Porter Stemmer

- Rule 1: Cleanup

Sr. No.	Rule	Condition Verified	Condition Not Verified
1.	at → ate	conflat(ed) → conflate	-
2.	bl → ble	troubl(ing) → trouble	-
3.	(*d & ! (*l or *s or *z)) → single letter	hopp(ing) → hop tann(ed) → tan	fall(ing) → fall
4.	(m=1 & *o) ing → e	fil(ing) → file	fail → fail

Porter Stemmer

- Rule 1: Part C

  Y Elimination (*V*)Y → I

  Condition verified: Happy → Happi

  Condition not verified: sky → sky

Porter Stemmer

- Rule 2: Derivational Morphology I

Sr. No.	Rule	Example
1.	(m > 0) ational → ate	relational → relate
2.	(m > 0) ization → ize	generalization → generalize
3.	(m > 0) biliti → ble	sensibiliti → sensible

Porter Stemmer

- Rule 3: Derivational Morphology II

Sr. No.	Rule	Example
1.	(m > 0) icate → ic	duplicate → duplic
2.	(m > 0) full → Φ	hopeful → hope
3.	(m > 0) ness → Φ	goodness → good

Porter Stemmer

- Rule 4: Derivational Morphology III

Sr. No.	Rule	Example
1.	(m > 0) ance → Φ	allowance → allow
2.	(m > 0) ent → Φ	dependent → depend
3.	(m > 0) ive → Φ	effective → effect

Porter Stemmer

- Rule 5: Cleanup
  - Rule 5a

Sr. No.	Rule	Example
1.	(m > 1) e → Φ	probate → probat
2.	(m = 1 & !=*o) ness → Φ	goodness → good

  - Rule 5b
    (m>1 & *d & *l) → single letter
    Condition verified: controll → control
    Condition not verified: roll → roll

Limitations of Porter Stemmer

- **Produces Non-Dictionary Words:** removes suffixes using **rules**, not a dictionary. As a result, the output may **not be a valid English word**.

  Example:
  university → univers
  communication → commun

- **Over-Stemming:** occurs when the stemmer **removes too much**, causing **different words with different meanings to be reduced to the same stem**.

  Example:
  organization → organ
  organ → organ

Limitations of Porter Stemmer

- **Under-Stemming:** occurs when the stemmer **fails to remove enough suffixes**, leaving related words with **different stems**.

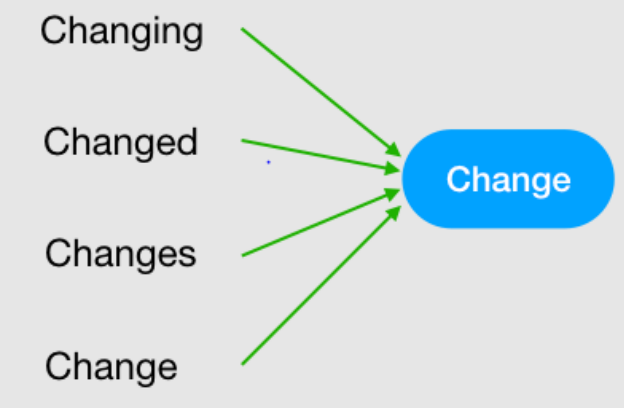
  Example:
  analysis → analysi
  analyst → analyst

- **Language Dependent**
  - The Porter Stemmer is **designed specifically for the English language**.
  - Rules are based on **English morphology**
  - Cannot be directly applied to other languages

LEMMATIZATION

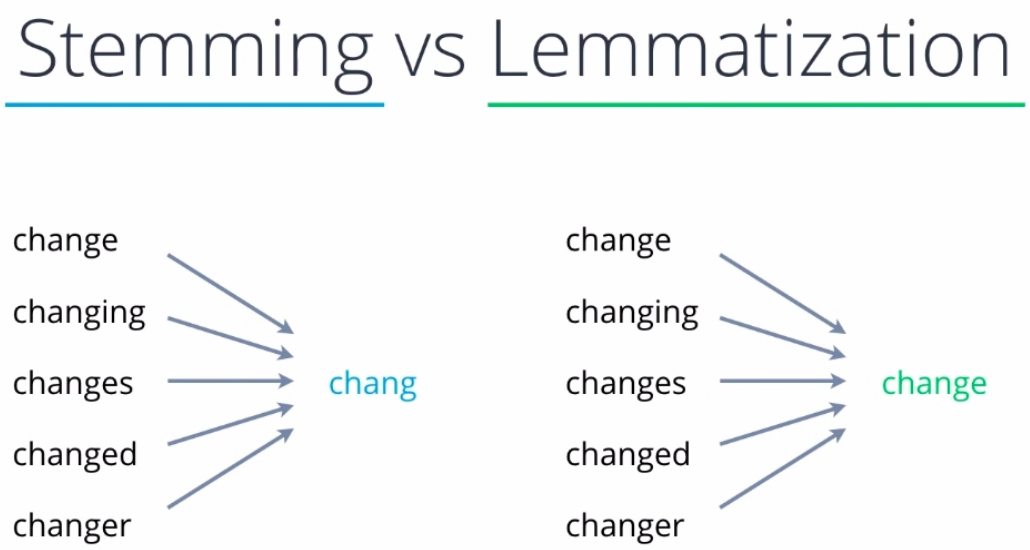
Lemmatization

- Text normalization technique that switches any kind of a word to its base root mode.
- Refers to the act of grouping together words that have the same root or lemma but have different inflections or derivatives of meaning so they can be analyzed as one item.
- The process of lemmatization seeks to get rid of inflectional suffixes and prefixes for the purpose of bringing out the word's dictionary form.



Lemmatization

- While stemming and lemmatization both focus on attempting to reduce the inflectional form of each word into a common base or root, they are not the same.
- In stemmin ... n prefixes and suffixes tha
- Lemmatizat ... f the word and link it to its
- Lemmatizat ... e converting into root-fo



Lemmatization

- Stemming tends to be a faster process than lemmatization because it chops words without knowing the context of the word in the sentences which they are in.
- Lemmatization, on the other hand, is a slower process than stemming, it knows the context of the word before proceeding.
- While stemming is a rule-based approach, lemmatization is a dictionary-based approach.
- The process of stemming also has a lower degree of accuracy as compared to lemmatization.

Lemmatization Techniques

- **Rule Based Lemmatization**
  - involves the application of predefined rules to derive the base or root form of a word.
  - relies on linguistic rules and patterns.
  - **Rule:** For regular verbs ending in “-ed,” remove the “-ed” suffix.
  - Example:
    - Word: “walked”
    - Rule Application: Remove “-ed”
    - Result: “walk
  - This approach extends to other verb conjugations, providing a systematic way to obtain lemmas for regular verbs.
  - may not cover all linguistic nuances
  - it serves as a transparent and interpretable method for deriving base forms in many cases.

Lemmatization Techniques

- **Dictionary-Based Lemmatization**
  - relies on predefined dictionaries or lookup tables to map words to their corresponding base forms or lemmas.
  - Each word is matched against the dictionary entries to find its lemma.
  - This method is effective for languages with well-defined rules.
  - Suppose we have a dictionary with lemmatized forms for some words:
    - ‘running’ -> ‘run’
    - ‘better’ -> ‘good’
    - ‘went’ -> ‘go’
  - When we apply dictionary-based lemmatization to a text like “I was running to become a better athlete, and then I went home,” the resulting lemmatized form would be: “I was run to become a good athlete, and then I go home.”

Lemmatization Techniques

- **Machine Learning-Based Lemmatization**
  - leverages computational models to automatically learn the relationships between words and their base forms.
  - machine learning models, such as neural networks or statistical models, are trained on large text datasets to generalize patterns in language.

Lemmatization - Advantages

- **Improves text analysis accuracy:** Lemmatization helps in improving the accuracy of text analysis by reducing words to their base or dictionary form. This makes it easier to identify and analyze words that have similar meanings.
- **Reduces data size:** Since lemmatization reduces words to their base form, it helps in reducing the data size of the text, which makes it easier to handle large datasets.
- **Better search results:** Lemmatization helps in retrieving better search results since it reduces different forms of a word to a common base form, making it easier to match different forms of a word in the text.

Lemmatization - Disadvantages

- **Time-consuming:** Lemmatization can be time-consuming since it involves parsing the text and performing a lookup in a dictionary or a database of word forms.
- **Not suitable for real-time applications:** Since lemmatization is time-consuming, it may not be suitable for real-time applications that require quick response times.
- **May lead to ambiguity:** Lemmatization may lead to ambiguity, as a single word may have multiple meanings depending on the context in which it is used. In such cases, the lemmatizer may not be able to determine the correct meaning of the word.

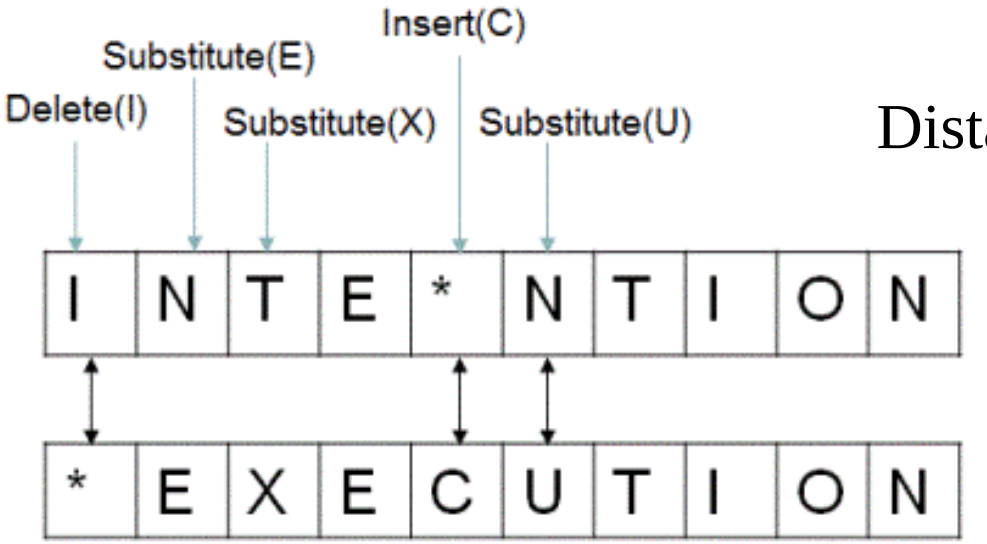
EDIT DISTANCE

Edit Distance

- Also known as Levenshtein distance
- is a string metric
- a way of quantifying how dissimilar two strings (e.g., words) are to one another, that is measured by counting the minimum number of operations (insertions, deletions, or substitutions) required to transform one string into the other.
- The smaller the edit distance, the more similar the two words are.

Edit Distance

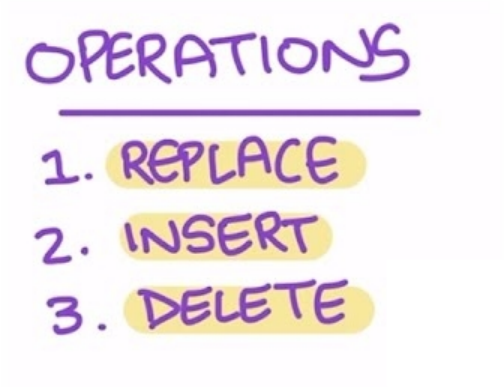
- Example: word1 = "intention", word2 = "execution"

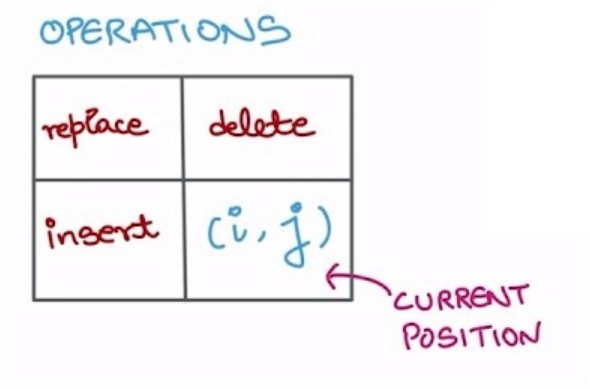


Distance = 5

Edit Distance

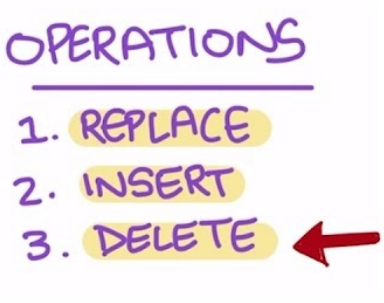
- Example: word1 = "replace", word2 = "delete"

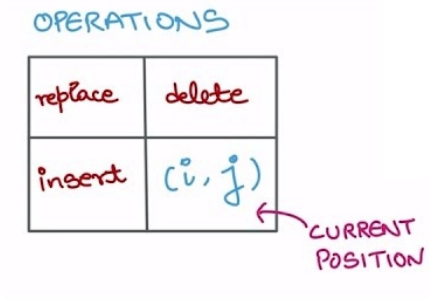




Edit Distance

- Example:
  word1 = "replace",
  word2 = "delete"





word 1

word 2	"" 0	r 1	e 2	p 3	l 4	a 5	c 6	e 7
""	0	1	2	3	4	5	6	7
1 d	1	1	2	3	4	5	6	7
2 e	2	2	1	2	3	4	5	6
3 l	3	3	2	2	2	3	4	5
4 e	4	4	3	3	3	3	4	4
5 t	5	5	4	4	4	4	4	5
6 e	6	6	5	5	5	5	5	4

Edit Distance

- A challenge is that its value will be sensitive only to the number of differences, while being insensitive to the length of the strings being compared.
- The next challenge is setting the Levenshtein distance threshold.

COLLOCATION

Collocation

- Collocations of a given word are statements of the habitual or customary places of that word.
- Collocations are phrases or expressions containing multiple words, that are highly likely to co-occur.
- For example — ‘social media’, ‘machine learning’ etc
- Collocations help in fetching the two or three words that are highly likely to co-occur around the themes.
  - These two or three words that occur together are also known as BiGram and TriGram.
- But Bigrams and Trigrams may not give meaningful phrases.
  - Example: He is an active user in Social Media.
  - Bigrams are ‘He is’, ‘is an’, ‘an active’, ‘active user’, ‘user in’, ‘in Social’, ‘Social Media’
  - Collocations are ‘active user’ and ‘Social Media’

Collocation

- Collocations are characterized by limited compositionality.
  - A natural language expression is compositional if the meaning of the expression can be predicted from the meaning of the parts.
- Collocations are not fully compositional in that there is usually an element of meaning added to the combination.
  - Example: Strong Tea : Strong has acquired the meaning rich
- Idioms are the most extreme examples of non-compositionality.
  - Example: to kick the bucket or to hear it through the grapevine

Characteristics of Collocation

- Non-compositionality.
  - The meaning of a collocation is not a straightforward composition of the meanings of its parts.
  - Either the meaning is completely different from the free combination or there is a connotation or added element of meaning that cannot be predicted from the parts.
  - For example, white wine, white hair and white woman all refer to slightly different colors, so we can regard them as collocations.
- Non-substitutability.
  - We cannot substitute near-synonyms for the components of a collocation.
  - For example, we can’t say yellow wine instead of white wine even though yellow is as good a description of the color of white wine as white is (it is kind of a yellowish white).

Characteristics of Collocation

- Non-modifiability.
  - Many collocations cannot be freely modified with additional lexical material or through grammatical transformations.
  - This is especially true for frozen expressions like idioms.
  - For example, we can’t modify ‘frog’ in ‘to get a frog in one’s throat’ into ‘to get an ugly frog in one’s throat’ although usually nouns like ‘frog’ can be modified by adjectives like ‘ugly’.
  - Similarly, going from singular to plural can make an idiom ill-formed
  - for example in ‘people’ as ‘poor as church mice’.

Finding Collocations

Frequency
Mean and Variance
Pointwise Mutual Information

- Frequency
  - If two words occur together a lot, then that is evidence that they have a special function.
  - But selecting the most frequently occurring bigrams is not very interesting.
  - A simple heuristic approach: pass the candidate phrases through a POS filter which only lets through those patterns that are likely to be phrases.
  - Many collocations consists of two words that stand in a more flexible relationship to one another
    - Example: a. She knocked on his door.
      b. They knocked at the door.
      c. A man knocked on the metal front door.
    - The words that appear between ‘knocked’ and ‘door’ vary and the distance between the two words is not constant.
    - So a fixed phrase approach will not work here

Finding Collocations

- Mean and Variance
  - Look at the pattern of varying distance between two words.
  - The mean is simply the average offset.
  - The variance measures how much individual offsets deviate from the mean.
  - The sample deviation, the square root of the variance, is used to assess how variable the offset between two words is.
  - The mean and deviation characterize the distribution of distances between two words in a corpus.
    - A low deviation means that the two words usually occur at about the same distance.
    - Zero deviation means that the two words always occur at exactly the same distance.
    - High deviation indicates that the two words of the apir stand in no interesting relationship.

Finding Collocations

- **Pointwise Mutual Information (PMI)**
  - It is a statistical measure to calculate the association
  - Positive PMI values indicate that the words have a strong association, while negative values indicate that the words are unlikely to appear together.

$$
\begin{aligned}
I(x', y') &= \log_2 \frac{P(x'y')}{P(x')P(y')} \\
&= \log_2 \frac{P(x'|y')}{P(x')} \\
&= \log_2 \frac{P(y'|x')}{P(y')}
\end{aligned}
$$

WORD EMBEDDING

Language modelling

- Language modeling is the way of determining the probability of any sequence of words.
- Language modelings are classified as follows:
- **Statistical language modelings**
- **Neural language modelings:**
  - Neural language modeling gives better results than the classical methods both for the standalone models and when the models are incorporated into the larger models on the challenging tasks i.e. speech recognitions and machine translations.
  - One method of performing neural language modeling is by word embedding

Word Embedding

- Word embedding or word vector is defined as a numeric vector input that allows words with similar meanings to have the same representation.
- It can approximate meaning and represent a word in a lower dimensional space.
- Types of Word Embeddings
  - Term frequency-inverse document frequency (TF-IDF)
  - Bag of words (BOW)
  - Word2Vec
  - GloVe
  - BERT (Bidirectional encoder representations from transformers)

Word Embedding
Term frequency-inverse document frequency (TF-IDF)

- It comprises two metrics, namely term frequency (TF) and inverse document frequency (IDF).
- The term frequency (TF) score measures the frequency of words in a particular document. In simple words, it means that the occurrence of words is counted in the documents.
- The inverse document frequency or the IDF score measures the rarity of the words in the text. It is given more importance over the term frequency score because even though the TF score gives more weightage to frequently occurring words, the IDF score focuses on rarely used words in the corpus that may hold significant information.
- It does not capture the semantic meaning of words efficiently in a sequence.

Word Embedding
Term frequency-inverse document frequency (TF-IDF)

$$\textbf{tf-idf}_{i,j} = \text{Term Frequency}_{i,j} \times \text{Inverse Document Frequency}_i$$

Where,

$$\textbf{Term Frequency}_{i,j} = \frac{\text{Term i frequency in document j}}{\text{Total no. of terms in document j}}$$

$$\textbf{Inverse Document Frequency}_i = \log\left(\frac{\text{Total documents}}{\text{No. of documents containing term i}}\right)$$

What is TF-IDF?

A statistical measure of how relevant a word is to a document within a collection of documents.

TF Term Frequency

How many times a word appears in a single document.

More frequent = higher TF score.

Simplest form:
count of word / total words in document

×

IDF Inverse Document Frequency

How rare a word is across ALL documents.

Rare words = high score
Common words = near 0

Formula uses logarithm of document ratios.

$$\text{TF-IDF}(\text{word}, \text{doc}) = \text{TF}(\text{word}, \text{doc}) \times \text{IDF}(\text{word})$$

Origin & Use Cases

Origin

Originally designed for document search and information retrieval.

How it works:
A query is run and the system finds the most relevant documents by weighting rare words more heavily than common ones.

Example query: *"The bug"*
→ "bug" is rare → high IDF → more weight
→ "the" is common → IDF ≈ 0 → ignored

Applications

Information Retrieval

Rank documents by query relevance

Keyword Extraction

Surface the most important terms

Machine Learning

Numeric features from raw text

Text Analysis

Measure topic importance across docs

Step 1: Term Frequency (TF)

TF(word, doc) = count of word in doc / total words in doc

Our 3 sample documents (Query Q = "The cat")

D1
"The cat is on the mat."
6 words

D2
"My dog and cat are the best."
7 words

D3
"The locals are playing."
4 words

TF scores for "the" and "cat":

Word	D1 (6 words)	D2 (7 words)	D3 (4 words)
"the"	2/6 = 0.33	1/7 = 0.14	1/4 = 0.25
"cat"	1/6 = 0.17	1/7 = 0.14	0/4 = 0.00

Step 2: Inverse Document Frequency (IDF)

```
IDF(word) = log(totaldocuments / documentscontaining word)
```

If a word appears in every document → log(1) = 0. If a word appears in only 1 of many → high score.

Rarity spectrum:

IDF = 0 (appears in all docs) **IDF high (appears in few docs)**

IDF calculation for "the" and "cat" (3 documents total):

Word	Appears in	Calculation	IDF Score
"the"	D1, D2, D3 (all 3)	log(3/3) = log(1)	*0.00 ← very common*
"cat"	D1, D2 (2 of 3)	log(3/2) = log(1.5)	**0.18 ← rarer word**

Step 3: TF-IDF Scores & Document Ranking

$$\text{TF-IDF}(\text{word}, \text{doc}) = \text{TF} \times \text{IDF}$$

Final TF-IDF scores:

Word	Document	TF	IDF	TF-IDF
"the"	D1	0.33	0.00	0.00
"the"	D2	0.14	0.00	0.00
"the"	D3	0.25	0.00	0.00
"cat"	D1	0.17	0.18	**0.0306** ★
"cat"	D2	0.14	0.18	**0.0252**
"cat"	D3	0.00	0.00	0.00

Document Ranking

Q = "The cat"

1st	D1 — avg 0.0153
2nd	D2 — avg 0.0126
3rd	D3 — avg 0.0000

💡 *"the" scores 0 in every document because it appears in ALL 3 — it carries no discriminative value.*

TF-IDF in Machine Learning

TF-IDF converts text into numeric vectors — enabling machine learning algorithms to process language.

Raw Text

"The cat is on the mat"

Tokenize

Split into words

TF-IDF Score

Score each token

Feature Vector

[0, 0.17, 0, 0.03...]

ML Model

Train / Predict

Classification

Each document becomes a TF-IDF vector. Models like Naive Bayes or SVM use these to classify topics, sentiment, or spam.

Search & Retrieval

Rank documents by cosine similarity between the query vector and document TF-IDF vectors.

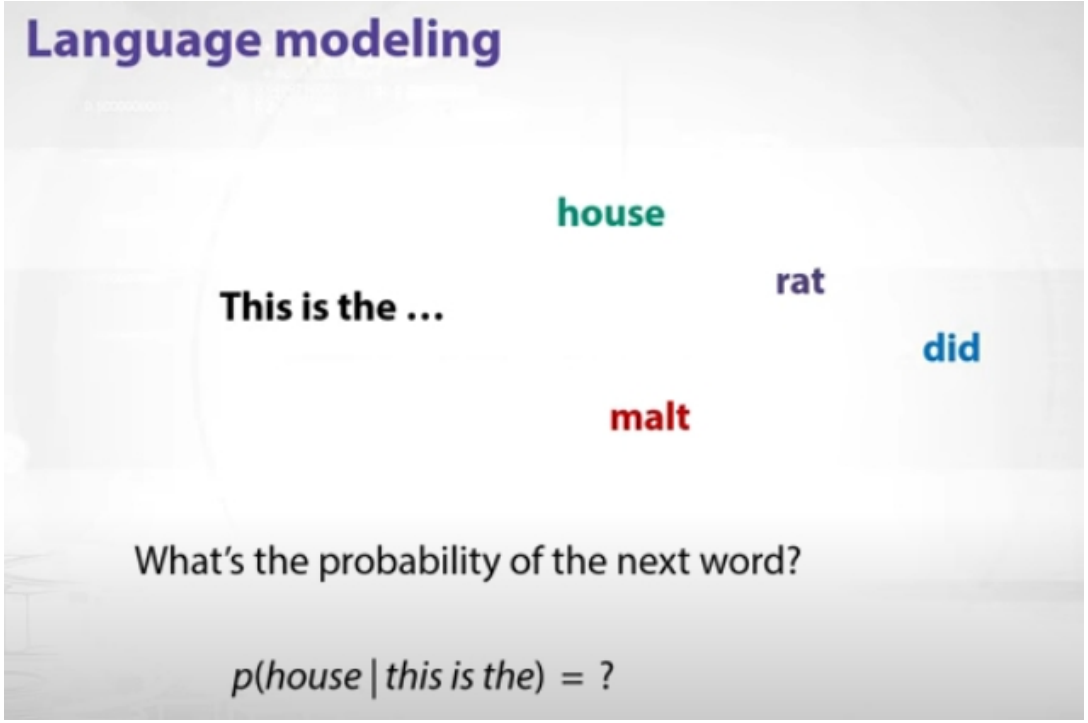
Clustering

Group similar documents using TF-IDF vectors as input to K-Means or hierarchical clustering algorithms.

N - GRAM LANGUAGE MODEL

Language modelling

- Language modeling is the way of determining the probability of any sequence of words.



Language modelling

- Language modeling is the way of determining the probability of any sequence of words.
- Language modelings are classified as follows:
- **Statistical language modelings**
- **Neural language modelings**

Language modelling

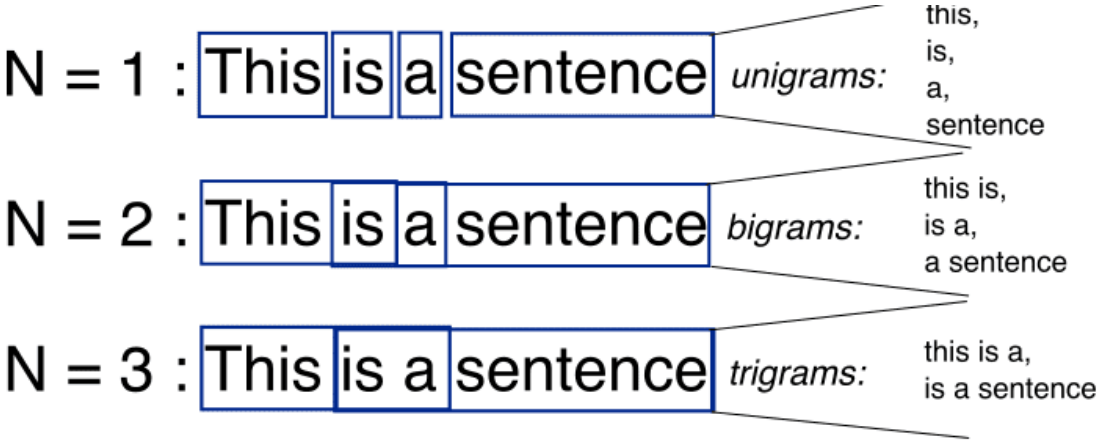
- Language modeling is the way of determining the probability of any sequence of words.
- Language modelings are classified as follows:
- **Statistical language modelings**:
  - probabilistic model predicts the next word in a sequence.
  - For example N-gram language modeling.
  - This modeling can be used for disambiguating the input.
- **Neural language modelings**

Language modelling

- Language modeling is the way of determining the probability of any sequence of words.
- Language modelings are classified as follows:
- **Statistical language modelings**
- **Neural language modelings:**
  - Neural language modeling gives better results than the classical methods both for the standalone models and when the models are incorporated into the larger models on the challenging tasks i.e. speech recognitions and machine translations.
  - One method of performing neural language modeling is by word embedding

N-gram modelling

- N-gram is a sequence of the N-words in the modeling of NLP.



N-gram modelling

This is the house that Jack built.
This is the malt
That lay in the house that Jack built.
This is the rat,
That ate the malt
That lay in the house that Jack built.
This is the cat,
That killed the rat,
That ate the malt
That lay in the house that Jack built.

N-gram modelling

p (house | this is the) = ?

This is the house *that Jack built.*
This is the malt
That lay in the house that Jack built.
This is the rat,
That ate the malt
That lay in the house that Jack built.
This is the cat,
That killed the rat,
That ate the malt
That lay in the house that Jack built.

N-gram modelling

p (house | this is the)

= p (this is the house) / p (this is the)

= 1 / 4

This is the house *that Jack built.*
This is the malt
That lay in the house that Jack built.
This is the rat,
That ate the malt
That lay in the house that Jack built.
This is the cat,
That killed the rat,
That ate the malt
That lay in the house that Jack built.

N-gram modelling

p (Jack | that)

= ?

This is the house that Jack built.
This is the malt
That lay in the house that Jack built.
This is the rat,
That ate the malt
That lay in the house that Jack built.
This is the cat,
That killed the rat,
That ate the malt
That lay in the house that Jack built.

N-gram modelling

- Predict probability of sequence of words is given by

$$\mathbf{w} = (w1\ w2\ w3\ \ldots\ wk)$$

  - Chain rule

$$p\ (\mathbf{w}) = p(w1)\ p(w2|w1) \ldots\ p(wk|w1w2 \ldots w(k-1))$$

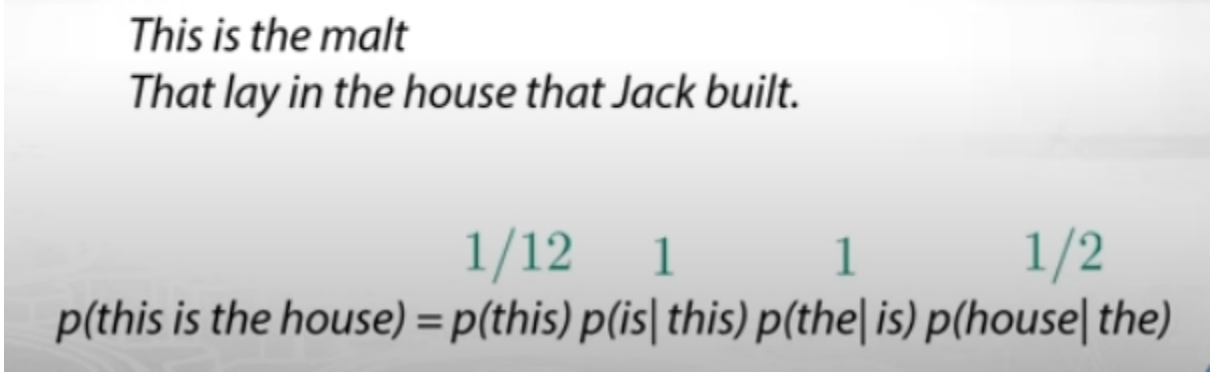
Start token

End token

  - Markov assumption

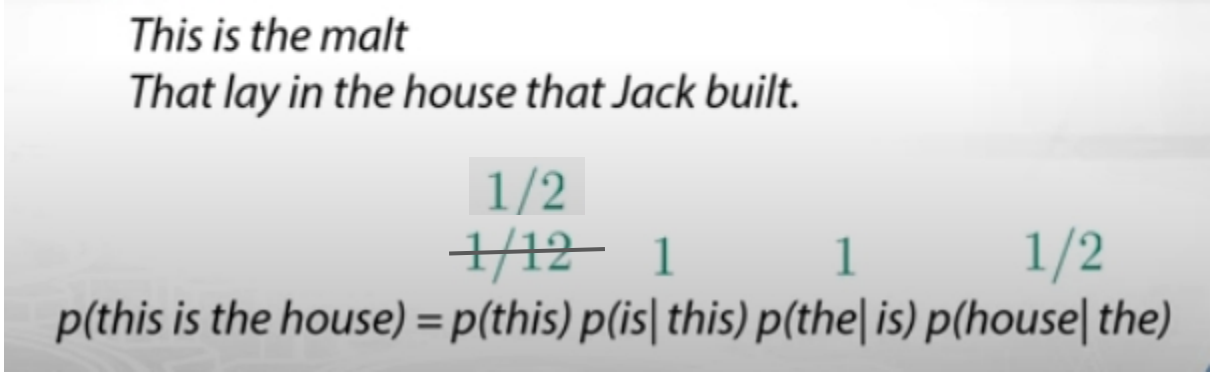
$$p\ (wk|w1 \ldots w(k-1)) = p\ (wk|w(i-n+1) \ldots w(k-1))$$

N-gram modelling



As per the toy corpus, the 1st word could be this or that.
Thereby not spreading the probability across all probability, and sticking to the words that would likely be in the start. Thus we can consider the fake start token.

N-gram modelling



As per the toy corpus, the 1st word could be this or that.
Thereby not spreading the probability across all probability, and sticking to the words that would likely be in the start. Thus we can consider the fake start token.

N-gram modelling

The probability is not normalized across the length.

$$p(\text{this}) + p(\text{that}) = 1$$
$$p(\text{this this}) + p(\text{this is}) + \ldots\ldots + p(\text{built built}) = 1$$

Thus fake tokens need to be added in the end.

Consider
dog
dog cat tiger
cat dog cat

p(cat dog cat) = ?

Unknown Words: Open Versus Closed Vocabulary Tasks

- If all the words that can occur is known well in advance then the vocabulary size V is known in advance.
- The closed vocabulary assumption is the assumption that there are no unknown lexicons.
- But since the number of unseen words grow constantly, we can’t possibly know in advance exactly how many are there.
  - The model needs to consider this aspect
- This unseen events of unknown words is known as Out of Vocabulary (OOV) words.
- The percentage of OOV words that appear in the test set is called the OOV rate.
- An open vocabulary system is one in which we model these potential unknown words in the test set by adding a pseudo word called <UNK> whose probabilities are trained.

Evaluating N grams: Perplexity

- The best way to evaluate the performance of a language model is to embed it in an application and measure the total performance of the application.
- Such end to end evaluation is called extrinsic evaluation
  - Very expensive - Evaluating time is large
- An intrinsic evaluation metric can be used to measure the quality of a model independent of any application. For N-gram models, commonly used intrinsic evaluation metric is Perplexity.
- Improvement in perplexity does not guarantee improvement in end to end metric but it often correlates with such improvements.
- The intuition of perplexity is that given two probabilistic models, the better model is the one that has a tighter fit to the test data or that better predicts the details of the test data.

Evaluating N grams: Perplexity

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

For bigrams:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

The higher the conditional probability of the word sequence, the lower the perplexity.